

"Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it."
- Brian W. Kernighan

CSE341 Programming Languages

Gebze Technical University Computer Engineering Department
2024-2025 Fall Semester

Procedures

© 2012-2024 Yakup Genç

Largely adapted from V. Shmatikov, J. Mitchell and R.W. Sebesta

1

Procedures vs. Functions

- Function:
 - no side effect
 - return a value
 - Function call: expression
- Procedure:
 - side effect, executed for it
 - no return value
 - Procedure call: statement
- No clear distinction made in most languages
 - C/C++: void
 - Ada/FORTRAN/Pascal: procedure/function

October 2024

CSE341 Lecture – Procedures

2

2

Syntax

- Terminology:
 - body
 - specification interface
 - name
 - type of return value
 - parameters (names and types)

```
int f(int y) {
    int x;
    x=y+1;
    return x;
}
```

f : int → int
f : int ⊗ float → int

```
int f(int y) //declaration
int f(int y){ //definition
    int x;
    x=y+1;
    return x;
}
```

October 2024

CSE341 Lecture – Procedures

3

3

Procedure Call

Caller:

Imp new

```
...
f(a);
...
```

Callee:

```
int f(int y){
    → int x;
    → if (y==0) return 0;
    → x=y+1;
    → return x;
}
```

- Control transferred from caller to callee, at procedure call
- Transferred back to caller when execution reaches the end of body
- Can **return** early

October 2024

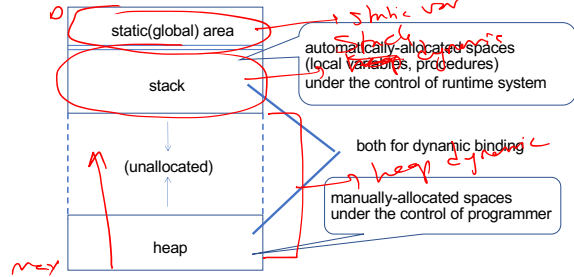
CSE341 Lecture – Procedures

4

4

Environment

- Environment: binding from names to their attributes



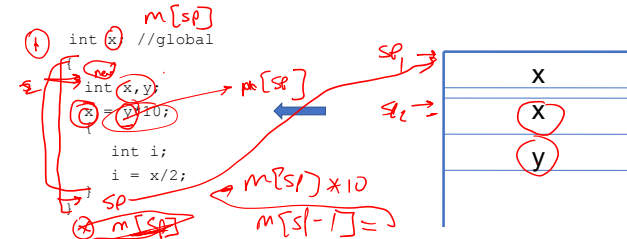
October 2024

CSE341 Lecture – Procedures

5

Activation Record for Nested Blocks

- Activation record: memory allocated for the local objects of a block
 - Entering a block: activation record allocated
 - Exit from inner block to surrounding block: activation record released



October 2024

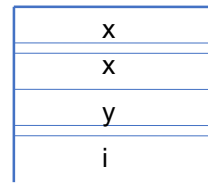
CSE341 Lecture – Procedures

6

Activation Record for Nested Blocks

```
int x; //global
{
  int x, y;
  x = y*10;
  {
    int i;
    i = x/2;
  }
}
```

x: nonlocal variable,
in the surrounding
activation record



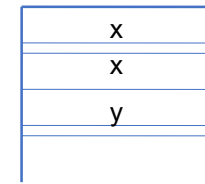
October 2024

CSE341 Lecture – Procedures

7

Activation Record for Procedures

```
int x; //global
void B(void) {
  int i;
  i = x/2;
}
void A(void) {
  int x, y;
  x = y*10;
  B();
}
main() {
  A();
  return 0;
}
```



October 2024

CSE341 Lecture – Procedures

8

Activation Record for Procedures

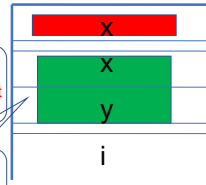
```

int x; //global
void B(void) {
  int i;
  i = x/2;
}
void A(void) {
  int x,y;
  x = y*10;
  B();
}
main() {
  A();
  return 0;
}

```

x: global variable in
defining environment

Need to retain
information in
calling environment



October 2024

CSE341 Lecture – Procedures

9

Activation Record for Procedures

```

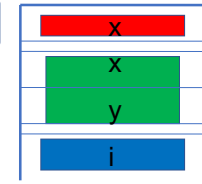
int x; //global
void B(void) {
  int i;
  i = x/2;
}
void A(void) {
  int x,y;
  x = y*10;
  B();
  A();
  return 0;
}
main() {
  A();
  return 0;
}

```

i: local variable in
called environment

x: global variable in
defining environment

x,y: local variable in
calling environment



October 2024

CSE341 Lecture – Procedures

10

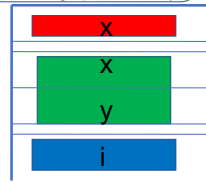
Activation Record for Procedures

```

int x; //global
void B(void) {
  int i;
  i = x/2;
}
void A(void) {
  int x,y;
  x = y*10;
  B();
}
main() {
  A();
  return 0;
}

```

Can only access global variables in
defining environment
No direct access to the local variables in the
calling environment
(Need to communicate through parameters)



October 2024

CSE341 Lecture – Procedures

11

Procedure Call

• Caller:

```

...
f(i);
...

```

actual parameter / argument

Callee:

```

int f(int a){
  ...;
  ...a...;
  ...;
}

```

formal parameter / argument

Parameter Passing Mechanisms:

- When and how to evaluate parameters
- How actual parameter values are passed to formal parameters
- How formal parameter values are passed back to actual parameters

October 2024

CSE341 Lecture – Procedures

12

Parameter Passing Mechanisms

- Pass/Call by Value
- Pass/Call by Reference
- Pass/Call by Value-Result
- Pass/Call by Name

October 2024

CSE341 Lecture – Procedures

13

13

Example

- What is the result?

```
void swap(int a, int b) {
    int temp;
    temp = a;
    a = b;
    b = temp;
}

main() {
    int i=1, j=2;
    swap(i,j);
    printf("i=%d, j=%d\n", i, j);
}
```

- It depends...

October 2024

CSE341 Lecture – Procedures

14

14

Pass by Value

- Caller:

```
...
f(i);
```

i

- Callee:

```
int f(int a) {
    ...a...;
}
```

a

- Most common one
- Replace formal parameters by the values of actual parameters
- Actual parameters: No change
- Formal parameters: Local variables (C, C++, Java, Pascal)

October 2024

CSE341 Lecture – Procedures

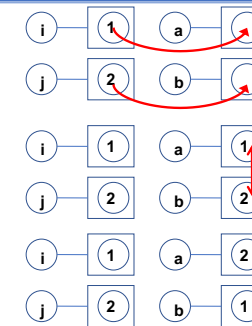
15

15

Example: Pass By Value

```
void swap(int a, int b) {
    int temp;
    temp = a;
    a = b;
    b = temp;
}

main() {
    int i=1, j=2;
    swap(i,j);
    printf("i=%d, j=%d\n", i, j);
}
```



October 2024

CSE341 Lecture – Procedures

16

16

Are these Pass-by-Value?

- C:

```
void f(int *p) { *p = 0; }
```

```
void f(int a[]) { a[0]=0; }
```
- Java:

```
void f(Vector v) { v.removeAll(); }
```

Yes!

October 2024

CSE341 Lecture – Procedures

17

17

Pass-by-Value: Pointers

C:

```
void f(int *p) { *p = 0; }
```

```
main() {
```

```
  int *q;
```

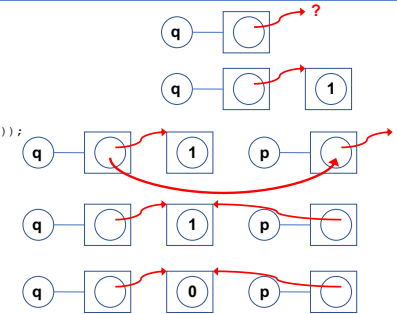
```
  q = (int *) malloc(sizeof(int));
```

```
  *q = 1;
```

```
  f(q);
```

```
  printf("%d\n", q[0]);
```

```
}
```



October 2024

CSE341 Lecture – Procedures

18

18

Pass-by-Value: Pointers

C:

```
void f(int *p) { p = (int *) malloc(sizeof(int)); *p = 0; }
```

```
main() {
```

```
  int *q;
```

```
  q = (int *) malloc(sizeof(int));
```

```
  *q = 1;
```

```
  f(q);
```

```
  printf("%d\n", q[0]);
```

```
}
```

- What happens here?

October 2024

CSE341 Lecture – Procedures

19

19

Pass-by-Value: Arrays

C:

```
void f(int p[]) { p[0] = 0; }
```

```
main() {
```

```
  int q[10];
```

```
  q[0]=1;
```

```
  f(q);
```

```
  printf("%d\n", q[0]);
```

```
}
```

- What happens here?

October 2024

CSE341 Lecture – Procedures

20

20

Pass-by-Value: Arrays

C:

```
void f(int p[]) { p=(int *) malloc(sizeof(int)); p[0] = 0; }
main() {
    int q[10];
    q[0]=1;
    f(q);
    printf("%d\n", q[0]);
}
```

- What happens here?

October 2024

CSE341 Lecture – Procedures

21

21

Pass-by-Value: Java Objects

Java:

```
void f(Vector v) { v.removeAll(); }

main() {
    Vector vec;
    vec.addElement(new Integer(1));
    f(vec);
    System.out.println(vec.size());
}
```

- What happens here?

October 2024

CSE341 Lecture – Procedures

22

22

Pass-by-Value: Java Objects

Java:

```
void f(Vector v) { v = new Vector(); v.removeAll(); }

main() {
    Vector vec;
    vec.addElement(new Integer(1));
    f(vec);
    System.out.println(vec.size());
}
```

- What happens here?

October 2024

CSE341 Lecture – Procedures

23

23

Pass by Reference

• Caller:

```
...
f(i);
...
```



Callee:

```
int f(int a){
    ...a...;
}
```



- Formal parameters become **alias** of actual parameters
- Actual parameters: changed by changes to formal parameters
- Examples:
 - Fortran: the only parameter passing mechanism
 - C++ (reference type, &) / Pascal (var)

October 2024

CSE341 Lecture – Procedures

24

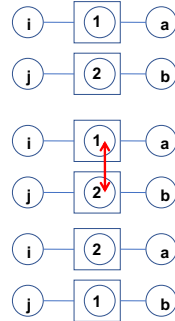
24

Example: Pass By Reference

C++ syntax. Not valid in C

```
void swap(int &a, int &b) {
    int temp;
    temp = a;
    a = b;
    b = temp;
}

main() {
    int i=1, j=2;
    swap(i,j);
    printf("i=%d, j=%d\n", i, j);
}
```



October 2024

CSE341 Lecture – Procedures

25

25

Pass-by-Reference: How to mimic it in C?

C:

```
void f(int *p) { *p = 0; }

main() {
    int q;
    q = 1;
    f(&q);
    printf("%d\n", q);
}
```

- It is really pass-by-value. Why?

October 2024

CSE341 Lecture – Procedures

26

26

It is really pass-by-value

C:

```
void f(int *p) { p = (int *) malloc(sizeof(int)); *p = 0; }

main() {
    int q;
    q = 1;
    f(&q);
    printf("%d\n", q);
}
```

October 2024

CSE341 Lecture – Procedures

27

27

Pass-by-Reference: C++ Constant Reference

C++:

```
void f(const int &p) {
    int a = p;
    p = 0;
}

main() {
    int q;
    q = 1;
    f(q);
    printf("%d\n", q);
}
```

const int &p
Error: expression must be a modifiable lvalue

- What happens here?

October 2024

CSE341 Lecture – Procedures

28

28

Pass-by-Reference: C++ Reference-to-Pointer

C++:

```
void f(int * &p) { *p = 0; }
main() {
    int *q;
    int a[10];
    a[0]=1;
    q=a;
    f(q);
    printf("%d, %d\n", q[0], a[0]);
}
```

00

- What happens here?

October 2024

CSE341 Lecture – Procedures

29

29

Pass-by-Reference: C++ Reference-to-Pointer

C++:

```
void f(int * &p) { p = new int; *p = 0; }
main() {
    int *q;
    int a[10];
    a[0]=1;
    q=a;
    f(q);
    printf("%d, %d\n", q[0], a[0]);
}
```

01

- What happens here?

October 2024

CSE341 Lecture – Procedures

30

30

Pass-by-Reference: C++ Reference-to-Array

C++:

```
void f(int (&p)[10]) { p[0]=0; }
main() {
    int *q;
    int a[10];
    a[0]=1;
    q = a;
    f(a);
    printf("%d, %d\n", q[0], a[0]);
}
```

00

- What happens here?

October 2024

CSE341 Lecture – Procedures

31

31

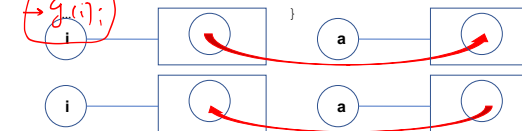
Pass by Value-Result

• Caller:

```
f(i);
```

Callee:

```
int f(int a) {
    ...a...;
}
```



- Combination of Pass-by-Value and Pass-by-Reference (Pass-by-Reference without aliasing)
- Replace formal parameters by the values of actual parameters
- Value of formal parameters are copied back to actual parameters

October 2024

CSE341 Lecture – Procedures

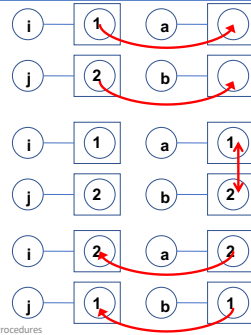
32

32

Example: Pass By Value-Result

```
void swap(int a, int b) {
    int temp;
    temp = a;
    a = b;
    b = temp;
}

main() {
    int i=1, j=2;
    swap(i,j);
    printf("i=%d, j=%d\n", i, j);
}
```



October 2024

CSE341 Lecture – Procedures

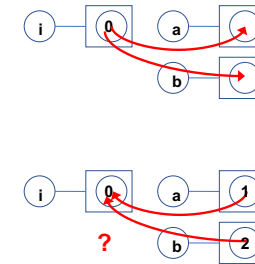
33

33

Unspecified Issues

```
void f(int a, int b) {
    a = 1;
    b = 2;
}

main() {
    int i=0;
    f(i,i);
    printf("i=%d\n", i);
}
```



October 2024

CSE341 Lecture – Procedures

34

34

Pass by Name

• Caller: Callee:

```
...           int f(int a) {
f(i);         ...a...;
...           }
```

- Actual parameters only evaluated when they are needed
- The same parameter can be evaluated multiple times
- Evaluated in calling environment
- Callee can change the values of variables used in the argument expression and hence change the expression's value
- Essentially equivalent to normal order evaluation
- Example:
 - Algol 60
 - Not adopted by any major languages due to implementation difficulty

October 2024

CSE341 Lecture – Procedures

35

35

Evaluation Strategy Revisited

- Strict Evaluation
 - the arguments to a function are always evaluated completely before the function is applied
 - eager evaluation
- Non-strict Evaluation
 - the arguments to a function are not evaluated unless they are actually used in the evaluation of the function body
 - short-circuit & lazy evaluation

October 2024

CSE341 Lecture – Procedures

36

36

Strict Evaluation Revisited

- **Applicative order:** the arguments of a function are evaluated from left to right
- **Call by value:** the argument expression is evaluated, and the resulting value is bound to the corresponding variable in the function
- **Call by reference:** a function receives an implicit reference to a variable used as argument
- **Call by sharing (or object):** differing from call-by-reference in that assignments to function arguments within the function are not visible to the caller
- **Call by value-result (or copy-restore):** a special case of call-by-reference where the provided reference is unique to the caller (Fortran & in multiprocessing context)

October 2024

CSE341 Lecture – Procedures

37

37

Non-strict Evaluation Revisited

- **Normal order (or leftmost outermost):** the outermost reducible expression is always reduced, applying functions before evaluating function arguments
- **Call by name:** the arguments to a function are not evaluated before the function is called, they are substituted (capture-avoiding) directly into the function body and then left to be evaluated whenever they appear in the function
- **Call by need:** a memoized version of call-by-name where, if the function argument is evaluated, that value is stored for subsequent uses. In pure functional programming, this produces the same results as call-by-name; when the function argument is used two or more times, call-by-need is almost always faster.
- **Call by macro extension:** similar to call-by-name, but uses textual substitution rather than capture-avoiding substitution

Memoization is an optimization technique used to speed up programs by having function calls avoid repeating the calculation of results for previously processed inputs

October 2024

CSE341 Lecture – Procedures

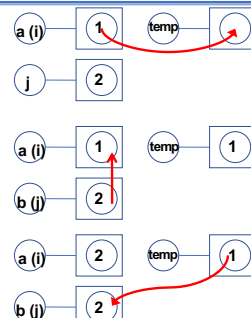
38

38

Example: Pass By Name

```
void swap(int a, int b) {
    int temp;
    temp = a;
    a = b;
    b = temp;
}

main() {
    int i=1, j=2;
    swap(i,j);
    printf("i=%d, j=%d\n", i, j);
}
```



October 2024

CSE341 Lecture – Procedures

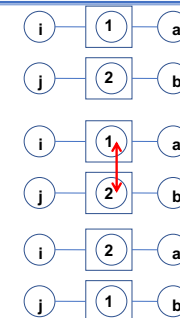
39

39

Example: Pass By Reference

```
void swap(int &a, int &b) {
    int temp;
    temp = a;
    a = b;
    b = temp;
}

main() {
    int i=1, j=2;
    swap(i,j);
    printf("i=%d, j=%d\n", i, j);
}
```



October 2024

CSE341 Lecture – Procedures

40

40

Pass-by-Name: Side Effects

```

int p[3]={3,2,1};           // call to swap(x,y)
int i;                      temp = x;

void swap(int a, int b) {    x = y;
    int temp;               y = temp;
    temp = a;               // call to swap(i,p[i])
    a = b;                  temp = i;
    b = temp;               i = p[i];
}                            p[i] = temp;

main() {
    i = 1;
    swap(i, p[i]);
    printf("%d, %d\n", i, p[i]);
}

```

What happens here?

before call ...
i=1 and x[1]=3
after call ...
i=3 and x[3]=1

October 2024

CSE341 Lecture – Procedures

41

41

Example: What's the use?

```

double sum(int j, int s, int e, double Ej) {
    double t;
    t = 0;
    for (j=s; j<=e; ) {
        t = t + Ej;
    }
    return t;
}

main() {
    int i=10;
    printf("%f\n", sum(i, 0, 20, x[i]*i));
}

```

October 2024

CSE341 Lecture – Procedures

42

42

Comparisons

- **Call by Value**
 - Efficient. No additional level of indirection.
 - Less flexible and less efficient without pointer.
 - (array, struct, union as parameters)
- **Call by Reference**
 - Require one additional level of indirection (explicit dereferencing)
 - If a parameter is not variable (e.g., constant), a memory space must be allocated for it, in order to get a reference.
 - Easiest to implement.
- **Call by Value-Result**
 - You may not want to change actual parameter values when facing exceptions.
- **Call by Name**
 - Lazy evaluation
 - Difficult to implement

October 2024

CSE341 Lecture – Procedures

43

43

Thank you for listening!

44